Fixup argument. After a collection no pointer may hold an old address, so we list every place a managed pointer can live and how each is handled.

1. **Shadow-stack slots.** These are the roots, and `fix_references` rewrites each to its object's new address.
2. **Pointer fields of live objects.** Rewritten for *every* live object, including ones that do not move, because an unmoved object can point at one that does.
3. **Pointers outside the heap.** Descriptors, literals, and `LO_EMPTY_STRING` fail the `managed()` range check and are left alone. They never move, so their addresses are already correct.
4. **Pointers held by the runtime itself.** There are none. String operations copy their inputs into a Rust `Vec` *before* allocating, so no managed pointer is held across a possible collection. `lo_cast_check` and `lo_instanceof` allocate nothing. The worklist and relocation table hold offsets, not pointers.
5. **The bump pointer.** `FREE` is reset to the end of the live prefix.

If these cover every location, nothing holds a stale address after `slide`. Dead objects are never visited, so their stale fields are never read.

3 Testing on Both Targets

LO stress tests (suite header-contract format) are in `tests/gc-stress/`. There are five, and each allocates about 3 million objects, keeps live data under a quarter of its heap, and was confirmed to fail on a copy of our runtime with the named bug planted.

Test	Invariant (planted bug it catches)
`gc_01_unmoved_anchor_fixup`	Fixup rewrites fields of unmoved live objects (fixup skips objects whose address is unchanged)
`gc_02_overlapping_slide`	A slide shorter than the object is overlap-safe (copying high to low, or the tail block first)
`gc_03_survivors_then_die`	Survivors are reclaimed once dead (mark bits left set)
`gc_04_deep_list_marking`	Marking uses a worklist (recursive marking overflows the stack on a very long link chain)
`gc_05_every_pointer_field`	Scan visits exactly the descriptor's offsets (first field only, last field skipped, contiguity assumed)

`run.sh` runs each program on WASM and native at the default heap, and on native at its declared heap. All five pass on both targets, and natively at their declared heaps. A WASM module has no environment, so `LO_HEAP_SIZE` cannot reach it and WASM always runs at 16 MiB. There, collections come from allocating past that heap.

Forced collections. LO source has no way to call `lo_gc_collect`, which the ABI reserves for tests, so these programs force collections by exhausting the heap instead. Each allocates about 3 million short-lived objects against its declared heap, so collections recur throughout the run while the program's live data is in a known shape. Direct `lo_gc_collect` calls are exercised by Rust tests, run natively and compiled to WASM, which `tests/gc-stress/README.md` maps to each stress-test category.

4 Worst Bug

[TODO: Confirm this is the bug the team wants to name, since it is the only one the git history records.]
The worst bug was at the collector's edge and differed by target. A heap-exhaustion abort inside a *string* operation (`lo_string_concat`, `lo_string_new`, `lo_string_reverse`) printed the right message on both targets but exited 137 on native and **134** on WASM. `wasmrun` derives the exit code from the trap backtrace and maps a trap to 137 only if a `lo_alloc*` frame is on it, and our string operations aborted from their own out-of-memory paths. We found it by running tiny-heap tests on WASM. The fix routes *every* out-of-memory abort through one `#[cold] #[inline(never)] lo_alloc_oom`. One case remains, since OOM inside `lo_string_repeat` still exits 120 on WASM because `wasmrun` matches that name first.

String and type operations. The five `lo_string_*` operations are UTF-8 aware (codepoint-aware reversal, byte-wise comparison, exit 120 on a negative repeat count) and follow the root discipline above. `lo_cast_check` and `lo_instanceof` walk the descriptor `parent` chain, with null short-circuits and exit 101 naming both classes.

Limitations. There is no heap-checker walk or GC log, only poisoning. Fixup costs a binary search per pointer rather than $O(1)$ forwarding, and the side tables reserve about twice the heap in address space.